

Database Development with PL/SQL INSY 8311



ADVENTIST UNIVERSITY
OF CENTRAL AFRICA

Instructor:

- Eric Maniraguha | ericmaniraguha2024@gmail.com | [LinkedIn Profile](#)



6h00 pm – 8h50 pm

- Monday A -G209
- Tuesday B-G209
- Wednesday E-G209
- Thursday F-G308

September 2024

Database Development with PL/SQL

Reference reading

- [Real Application Clusters \(RAC\) Articles](#)
- [Oracle SQL Queries: Basic Exercises with Solution](#)
- [Autonomicity in Oracle Database Management System](#)
- [The Toughest Challenges and Problems with Oracle Database](#)



Lecture 01 - Introduction to PL/SQL

What is PL/SQL?

What?

PL/SQL is a procedural relational database language developed by **Oracle Corporation** in earlier 90's.



PL/SQL – Procedural Language Structured Query Language.



Procedural language?

"PL/SQL allows the execution of a block of code at a time which increases its performance."

Course Overview



**Starting
out
with**

Oracle

This course provides an introduction to Oracle's database environment, including SQL, PL/SQL, and SQL*Plus. You will learn the fundamentals of these tools and languages, how they interact, and their applications in managing and manipulating relational databases.

**Covering Databases,
SQL, PL/SQL,
Developer Tools
and DBA**

Brief History of Oracle Database

The Oracle Database has evolved over more than 35 years of development, marked by several key milestones:

- **Founding (1977):** Larry Ellison, Bob Miner, and Ed Oates founded Software Development Laboratories, which later became Oracle Corporation.
- **First RDBMS (1979):** Oracle V2 was introduced as the first commercially available SQL-based Relational Database Management System (RDBMS).
- **Portable Database (1983):** Oracle Version 3, written in C, became the first RDBMS to run on various platforms, including mainframes, minicomputers, and PCs.
- **Concurrency and Scalability (1985-1996):**
 - **Version 4:** Introduced multiversion read consistency.
 - **Version 5:** Added client/server computing and distributed database support.
 - **Version 6:** Improved disk I/O, row locking, scalability, and introduced PL/SQL.
- **PL/SQL Enhancements (1992):** Oracle7 introduced PL/SQL stored procedures and triggers.
- **Object-Relational and Partitioning (1997):** Oracle8 supported object-relational features, new data types, and table partitioning.
- **Internet Computing (1999):** Oracle8i added native support for internet protocols and Java, designed for multitier environments.
- **Oracle RAC (2001):** Oracle9i introduced Real Application Clusters (RAC) and XML Database (XML DB), enabling multiple instances to access a single database and support for XML.
- **Grid Computing (2003):** Oracle Database 10g introduced grid computing, allowing virtualization of resources with a focus on self-management and self-tuning. Automatic Storage Management (ASM) simplified storage management.
- **Manageability and Automation (2007):** Oracle Database 11g improved manageability, diagnosability, and adaptability with enhanced features for information infrastructure and automation.
- **Cloud Computing (2013):** Oracle Database 12c was designed for the cloud with a Multitenant architecture, In-Memory column store, and support for JSON, optimizing IT resource usage and reducing costs.



What is SQL*Plus?

SQL*Plus is a versatile tool for interacting with Oracle databases, available in several interfaces:

- **Command-Line Interface:** Directly accessible via command-line terminals.
- **Windows GUI:** Provides a graphical interface on Windows platforms.
- **iSQL*Plus:** A web-based interface that allows for access through a web browser.

Key Features and Capabilities

1. Interactive and Batch Query Tool:

- **Interactive Mode:** Run SQL and PL/SQL commands interactively, with immediate feedback.
- **Batch Mode:** Execute scripts and commands in batch mode for automation and scheduled tasks.

2. Command Execution:

- **SQL Commands:** Perform data manipulation and retrieval using SQL.
- **PL/SQL Blocks:** Execute procedural code blocks.
- **SQL*Plus Commands:** Manage environment settings and format output.
- **Operating System Commands:** Run OS commands from within SQL*Plus.

3. Report Generation:

- **Formatting:** Customize the layout and appearance of query results.
- **Calculations:** Perform calculations and data processing on query results.
- **Output Options:** Export results to text files, display on-screen, or save in HTML format for web viewing.

4. Database Administration:

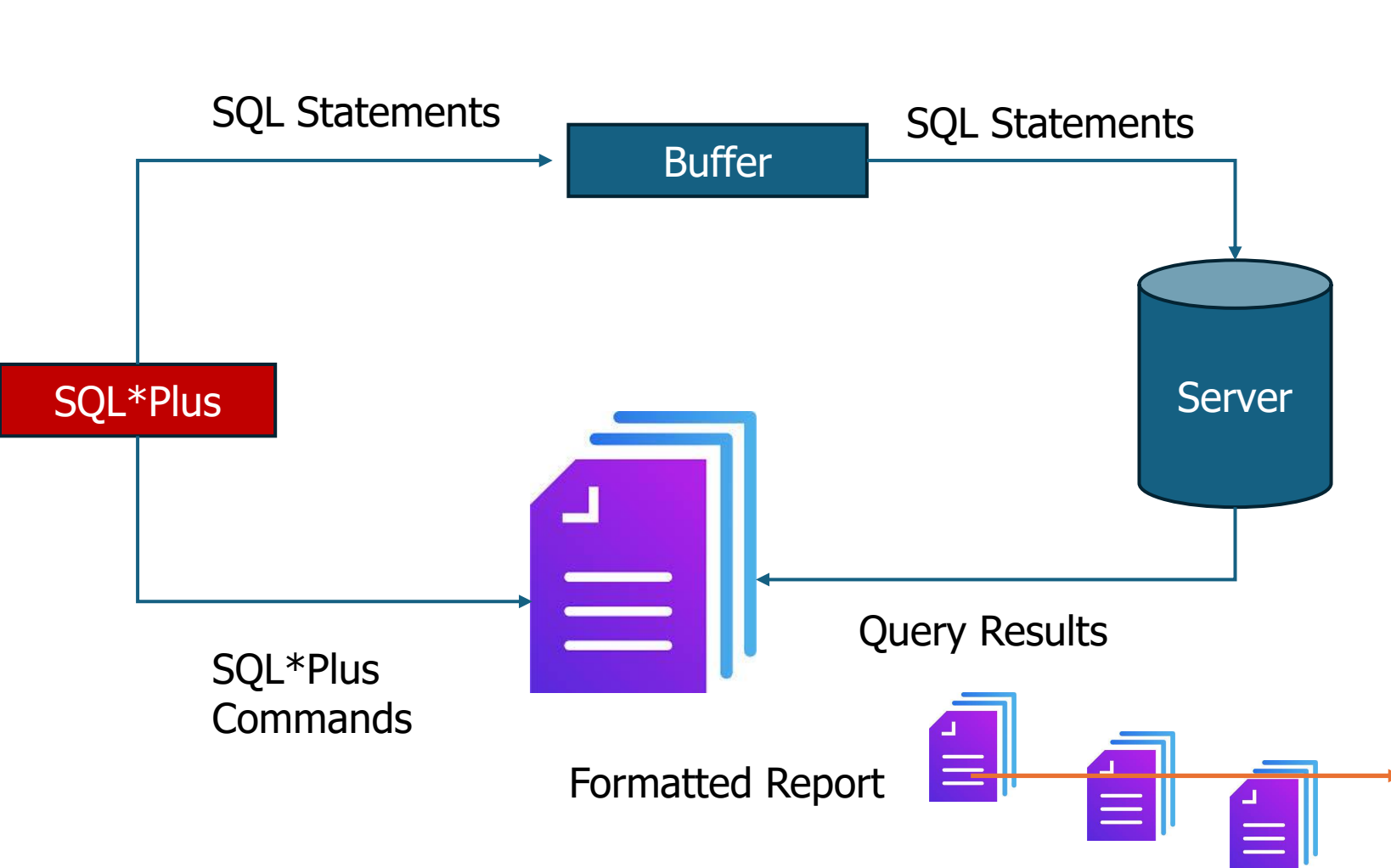
- **Table and Object Definitions:** Examine and manage database schema and objects.
- **Batch Scripts:** Develop and run scripts for repetitive tasks or database maintenance.

5. Dynamic Reporting:

- **HTML Output:** Generate HTML reports for browsing and sharing online.
- **iSQL*Plus:** Use the web-based interface for dynamic reporting and script execution from a web page.



SQL and SQL*Plus Interaction



Process Flow:

1. The user inputs **SQL statements** in **SQL*Plus**.
2. These statements are stored in the **buffer** temporarily.
3. The statements are sent from the buffer to the **server**, where they are processed.
4. The **query results** are returned to **SQL*Plus**.
5. **SQL*Plus** formats these results into a **report**.
6. The **formatted report** is then displayed to the user.



SQL*Plus - Usage

- **Starting SQL*Plus:**

- **Command-Line Interface:** Open your terminal or command prompt and type sqlplus, then enter your username and password.
- **Windows GUI:** Launch the SQL*Plus application from the start menu or shortcuts.
- **iSQL*Plus:** Access through a web browser by navigating to the iSQL*Plus URL.

- **Basic Commands:**

- **Connect to Database:**

CONNECT username/password@database

- **Run a SQL Query:**

SELECT * FROM employees;

- **Command Execution:**

SET SERVEROUTPUT ON;

- **Execute a PL/SQL Block:**

```
BEGIN
    DBMS_OUTPUT.PUT_LINE('Hello, World!');
END;
```

- **Generate a Report:**

SPOOL report.html SELECT * FROM employees; SPOOL OFF



SQL*Plus is essential for **database management, development, and reporting**, providing a robust set of tools for both interactive and automated tasks.

Oracle SQL (Structured Query Language)

- **Purpose:** SQL is a declarative language used to perform operations on data within a relational database. It focuses on querying, manipulating, and managing data.
- **Usage:** Oracle SQL is primarily used for:
 - **Data Manipulation:** Inserting, updating, deleting, and querying data in tables (e.g., **INSERT, SELECT, UPDATE, DELETE**).
 - **Data Definition:** Creating and modifying database structures like tables, views, and indexes (e.g., **CREATE, ALTER, DROP, TRUNCATE**).
 - **Data Control:** Managing access permissions (e.g., **GRANT, REVOKE**).
 - **Transaction Control:** Managing transactions (e.g., **COMMIT, ROLLBACK**).
- **Key Features:**
 - **Non-Procedural:** SQL focuses on *what* needs to be done (e.g., querying a database) rather than *how* it should be done.
 - **Single Statement Execution:** SQL executes individual queries or commands one at a time.
 - **Data-Centric:** SQL operates at the level of sets of data, allowing you to query multiple rows at once.
- **Example:**
 - `SELECT first_name, last_name FROM employees WHERE salary > 5000;`

Oracle PL/SQL (Procedural Language/SQL)

- **Purpose:** PL/SQL is Oracle's procedural extension to SQL, designed to enable procedural programming and control structures (like loops, conditionals, and error handling) along with SQL commands.
- **Usage:** PL/SQL is used for:
 - **Writing Programs:** Creating procedures, functions, packages, and triggers to execute complex logic.
 - **Control Structures:** Adding loops (FOR, WHILE), conditionals (IF-THEN-ELSE), and error handling (EXCEPTION block) to handle complex business logic.
 - **Batch Processing:** Running multiple SQL statements in sequence, using variables, cursors, and control statements.
 - **Improved Performance:** Avoids multiple round-trips between the application and the database by bundling SQL statements into a single block of code.
- **Key Features:**
 - **Procedural:** PL/SQL allows defining *how* tasks should be performed using control structures and procedural code.
 - **Block Structure:** PL/SQL code is organized into blocks that contain declarations, execution, and exception handling sections.
 - **Integration with SQL:** PL/SQL seamlessly integrates with SQL, allowing both procedural logic and SQL queries to be combined in one program.
 - **Functions and Procedures:** Allows you to define reusable logic in the form of functions and stored procedures.

Use of PL/SQL and Its Equivalents in Different Database Systems



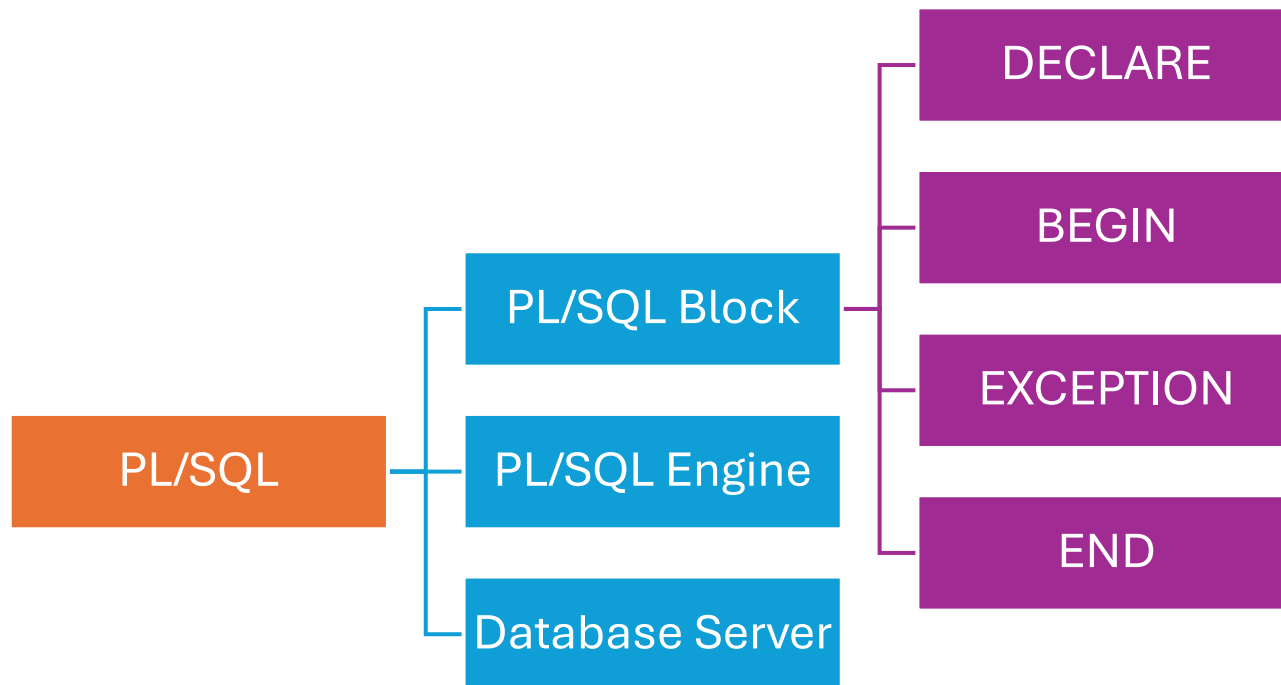
PL/SQL (Procedural Language/Structured Query Language) is specifically designed for Oracle databases. It is Oracle Corporation's procedural extension to SQL and is tightly integrated with Oracle's SQL engine.

Database System	Procedural Language	Description
Oracle	PL/SQL	Oracle's proprietary procedural extension to SQL.
PostgreSQL (pgAdmin)	PL/pgSQL	Procedural language designed for PostgreSQL.
Microsoft SQL Server	T-SQL (Transact-SQL)	Procedural extension used in Microsoft SQL Server.
MySQL	MySQL SQL/PSM	MySQL's procedural extension for SQL.

In summary, **PL/SQL is exclusive to Oracle databases**, and each database management system (DBMS) typically has its own procedural language.

The Architecture of PL/SQL (1/2)

PL/SQL (Procedural Language/Structured Query Language) is Oracle Corporation's procedural extension to SQL, designed specifically for **managing and manipulating data in Oracle databases**. It combines the data manipulation capabilities of SQL with the procedural programming features of languages like C or Java, allowing developers to write complex scripts that can execute multiple **SQL statements, control logic, handle exceptions**, and more **within a single program unit**.



Following are the different type of PL/SQL units.

- Anonymous Block
- Function
- Library
- Procedure
- Package Body
- Package Specification
- Trigger
- Type
- Type Body

Oracle PL/SQL Anonymous Block – Example

- **Example:**

```
-- Declare the PL/SQL block
DECLARE
  -- Declare a variable `v_salary` of type NUMBER with a precision of 8 digits and 2
  decimal places
  v_salary NUMBER(8,2);

-- Start the executable section of the PL/SQL block
BEGIN
  -- Execute a SELECT statement to retrieve the salary of the employee with employee_id
  = 1001
  -- Store the retrieved salary into the variable `v_salary`
  SELECT salary INTO v_salary FROM employees WHERE employee_id = 1001;

  -- Check if the value of `v_salary` is greater than 5000
  IF v_salary > 5000 THEN
    -- If the condition is true (i.e., `v_salary` is greater than 5000), output 'High salary'
    DBMS_OUTPUT.PUT_LINE('High salary');
  ELSE
    -- If the condition is false (i.e., `v_salary` is 5000 or less), output 'Low salary'
    DBMS_OUTPUT.PUT_LINE('Low salary');
  END IF;

-- End of the executable section
END;
```

Anonymous PL/SQL Block:

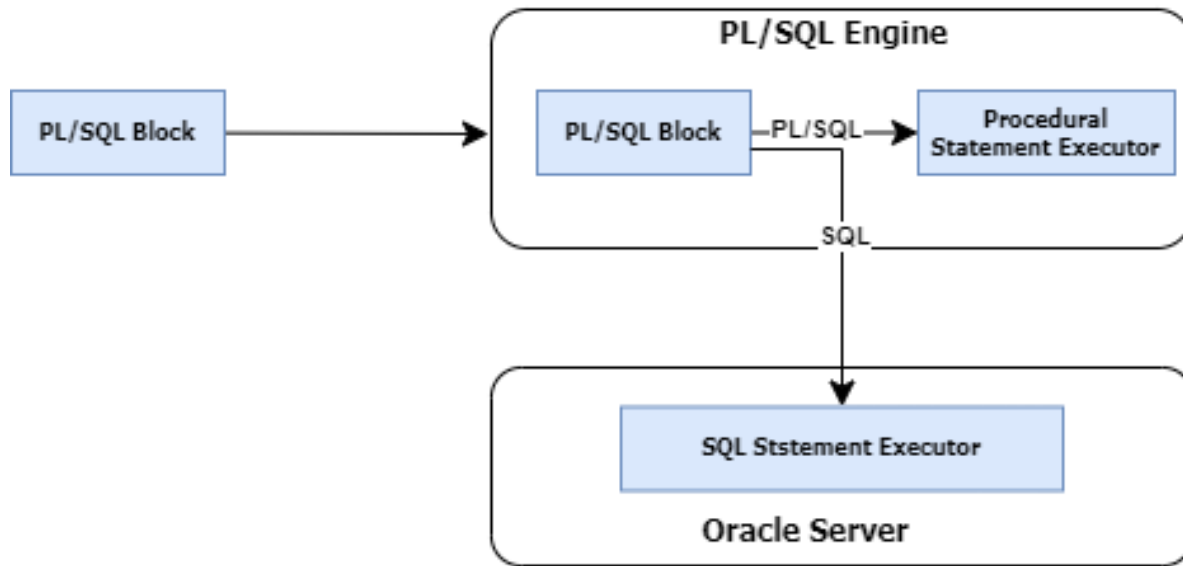
- **Not Named:** It is not stored with a name in the database.
- **Temporary:** Executed once, and then discarded.

Key Points

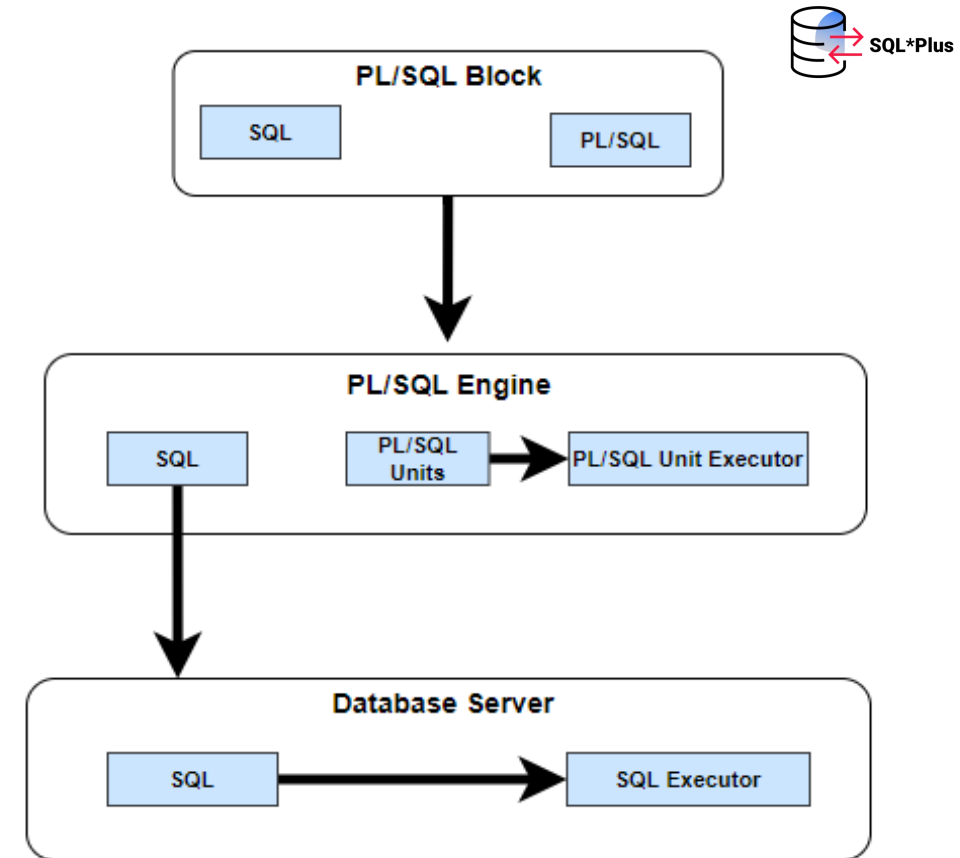
- **Variable Declaration:** v_salary is defined to store the salary of an employee.
- **Data Retrieval:** The SELECT INTO statement retrieves data from the database and stores it in the variable.
- **Conditional Logic:** The IF-THEN-ELSE construct is used to perform different actions based on the value of v_salary.
- **Output:** DBMS_OUTPUT.PUT_LINE is used to display messages based on the condition.



The Architecture of PL/SQL



1



2

The Architecture of PL/SQL - Explanation



1. PL/SQL Block:

- At the top, you have a **PL/SQL block**, which contains both **SQL** and **PL/SQL** code.
- **SQL** statements refer to standard SQL commands like **SELECT**, **INSERT**, **UPDATE**, etc.
- **PL/SQL** refers to procedural extensions for SQL, such as loops, conditions, and variable declarations.

2. PL/SQL Engine:

- The **PL/SQL engine** is responsible for processing the PL/SQL code.
- When a PL/SQL block is passed to the PL/SQL engine, it splits the **SQL** commands and **PL/SQL** logic into separate units.
- The **PL/SQL Units** are passed to the **PL/SQL Unit Executor** for processing.

3. SQL Execution:

- The SQL statements (from the PL/SQL block) are passed down to the **Database Server**.
- The database server contains the **SQL Executor**, which runs the SQL commands against the database.

The PL/SQL engine processes the procedural logic, and whenever a SQL statement is encountered, it hands that statement off to the database server to execute. The results from SQL execution are then returned to the PL/SQL engine to continue processing any remaining PL/SQL logic.

Key Components:

- **PL/SQL Block:** The block that contains both SQL statements and PL/SQL procedural code.
- **PL/SQL Engine:** Executes the PL/SQL logic and passes SQL commands to the database.
- **Database Server:** Executes SQL commands.

In summary, PL/SQL is processed in the PL/SQL engine, and SQL statements are processed in the database server. Both work together to execute your program or query efficiently.

PL/SQL Component description - Table

Component	Description
PL/SQL Block	▪ The fundamental unit of PL/SQL programming, containing the actual PL/SQL code. ▪ Divided into three sections: ○ Declarative (for variable declarations), ○ Execution (for processing logic and SQL statements), ○ Exception Handling (for managing errors). ▪ Interacts with the database using SQL instructions. ▪ Serves as the starting stage in PL/SQL architecture.
PL/SQL Engine	▪ The core component where the actual processing of PL/SQL code occurs. ▪ Separates PL/SQL units and SQL part from the input. ▪ Handles PL/SQL units, while the SQL part is sent to the database server for execution. ▪ Can be installed on both the database server and application server.
Database Server	▪ The crucial component that stores and manages the data. ▪ Interacts with the PL/SQL engine through SQL. ▪ Contains the SQL executor, which parses and executes SQL statements.



Why Use PL/SQL for Database Development? Key Features and Benefits (1/2)



Why Use PL/SQL for Database Development?

Key Features and Benefits (2/2)

Feature	Description	Example	
High Performance	PL/SQL is designed for high-performance transaction processing, optimizing database operations.	A PL/SQL block that retrieves and processes thousands of records faster than a standard SQL query.	
Code Reusability	Supports code reusability, allowing you to write modular code that can be reused across different programs.	Creating a procedure to calculate tax, which can be called whenever needed, instead of rewriting the tax calculation logic.	
Error Handling	Provides robust error handling by returning user-friendly error messages when errors occur.	Using EXCEPTION to catch a NO_DATA_FOUND error and return a custom message.	
Procedural Language Capability	Includes procedural language constructs like conditional statements (IF...ELSE) and loops (FOR loop).	Using an IF...ELSE statement to check a condition and a FOR loop to iterate over a range of values.	
Block Structure	Consists of nested blocks of code, where each block is a unit of task or logical module.	A block containing variable declarations, SQL statements, and an exception-handling section, all within a single logical unit.	
Better Performance	Oracle engine processes multiple SQL statements in a single block, reducing network traffic.	A PL/SQL block that executes multiple SQL operations (e.g., INSERT, UPDATE, DELETE) in a single transaction.	
Variable Declaration	Allows declaring variables that can be used and manipulated within the block.	Declaring a variable to store an employee's salary and then updating it within the same block.	
Looping and Conditional Statements	Supports loops (FOR, WHILE) and conditional statements (IF...ELSE) for complex control flows.	A FOR loop to iterate over a list of employee IDs and apply a bonus if the salary exceeds a certain amount.	
Reduce Network Traffic	Multiple SQL statements are processed as a single block, minimizing network interactions with the database.	Combining several INSERT operations in one PL/SQL block instead of executing them individually.	
Portable Applications	PL/SQL applications are portable across different operating systems without modification.	A PL/SQL procedure created on an Oracle database in Windows will work on Linux without changes.	

Simple Examples for Each Feature (1/5)

High Performance

```
BEGIN
-- Loop through each record from the 'employees' table where department_id is 10
FOR rec IN (SELECT * FROM employees WHERE department_id = 10) LOOP
    -- Output the first name of each employee in the result set
    DBMS_OUTPUT.PUT_LINE('Employee: ' || rec.first_name);
END LOOP;
END;
```

- **FOR employee_record IN (SELECT first_name FROM employees WHERE department_id = 10):**
This loop iterates through the result set of the query, which retrieves the first_name of employees in department 10.
- **DBMS_OUTPUT.PUT_LINE(employee_record.first_name);:**
This line prints the first_name of each employee to the output.

Make sure that DBMS_OUTPUT is enabled in your environment to see the output. You can enable it in SQL*Plus or SQL Developer by executing:

- **SET SERVEROUTPUT ON;**

Code Reusability

```
CREATE OR REPLACE PROCEDURE calculate_tax (p_salary IN NUMBER) IS
-- Declare a variable to store calculated tax
v_tax NUMBER;
BEGIN
-- Calculate 10% tax on the input salary
v_tax := p_salary * 0.10;

-- Output the calculated tax value
DBMS_OUTPUT.PUT_LINE('Tax: ' || v_tax);
END;
```

- **Procedure Name:** calculate_tax
- **Input Parameter:** p_salary (of type NUMBER)
- **Variable:** v_tax is declared to store the calculated tax.
- **Main Logic:** It calculates 10% of the salary ($p_salary * 0.10$) and assigns it to v_tax.
- **Output:** The calculated tax is printed using DBMS_OUTPUT.PUT_LINE.

Simple Examples for Each Feature (2/5)

Error Handling

```
BEGIN
-- Retrieve salary of employee with employee_id 999 and store in v_salary
SELECT salary INTO v_salary FROM employees WHERE employee_id = 999;

EXCEPTION
-- Handle case when no record is found for the given employee_id
WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No employee found with the given ID.');
```

END;

- **Main Logic:** The block tries to retrieve the salary of an employee with employee_id = 999 and store it in v_salary.
- **Exception Handling:** If no employee is found with that ID, the NO_DATA_FOUND exception is caught, and a message is printed using DBMS_OUTPUT.PUT_LINE.

Procedural language Capability

```
BEGIN
-- Check if the salary is greater than 5000
IF v_salary > 5000 THEN
    -- If salary is greater than 5000, print "High salary"
    DBMS_OUTPUT.PUT_LINE('High salary');
ELSE
    -- If salary is 5000 or less, print "Low salary"
    DBMS_OUTPUT.PUT_LINE('Low salary');
END IF;
END;
```

- **IF v_salary > 5000 THEN:**
This line checks if the variable v_salary (employee salary) is greater than 5000.
- **DBMS_OUTPUT.PUT_LINE('High salary');**
If the condition is true (salary > 5000), it prints "High salary" to the output.
- **ELSE:**
If the salary is less than or equal to 5000, the code executes the block in the ELSE statement.
- **DBMS_OUTPUT.PUT_LINE('Low salary');**
This line prints "Low salary" if the salary is 5000 or less.
- **END IF;**
Ends the conditional block.

Simple Examples for Each Feature (3/5)

Block Structure

```
DECLARE
  -- Declare a variable to store the salary
  v_salary NUMBER;
BEGIN
  -- Select the salary of the employee with ID 100 into the v_salary variable
  SELECT salary INTO v_salary FROM employees WHERE employee_id = 100;

  -- Check if the salary is greater than 5000 and print 'High salary' if true
  IF v_salary > 5000 THEN
    DBMS_OUTPUT.PUT_LINE('High salary');
  END IF;

  EXCEPTION
    -- Handle the case where no employee with the specified ID is found
    WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('No employee found');
END;
```

- **DECLARE:** Declares the variable `v_salary`.
- **BEGIN:** Executes the salary query.
- **IF:** Prints "High salary" if salary exceeds 5000.
- **EXCEPTION:** Catches `NO_DATA_FOUND` and prints "No employee found".

Better Performance

```
BEGIN
  -- Update salaries in department 10 by 10%
  UPDATE employees SET salary = salary * 1.10 WHERE department_id = 10;

  -- Delete employee with employee_id = 999
  DELETE FROM employees WHERE employee_id = 999;

  -- Commit both operations in one transaction
  COMMIT;
END;
```

- **Update:** Increases salaries by 10% for all employees in department 10.
- **Delete:** Removes the employee with `employee_id = 999`.
- **Commit:** Saves both changes in a single transaction to the database.

This block efficiently handles both operations together and ensures data consistency.

Simple Examples for Each Feature (4/5)

Variable Declaration

```
DECLARE
-- Declare and initialize bonus variable
v_bonus NUMBER := 1000;
-- Declare salary variable
v_salary NUMBER;
BEGIN
-- Retrieve the salary of the employee with ID 100
SELECT salary INTO v_salary FROM employees WHERE employee_id = 100;

-- Add bonus to the retrieved salary
v_salary := v_salary + v_bonus;

-- Print the new salary
DBMS_OUTPUT.PUT_LINE('New salary: ' || v_salary);
END;
```

- **Variable Declaration:**
 - **v_bonus:** Initialized to 1000.
 - **v_salary:** Declared to hold the employee's salary.
- **Retrieve Salary:**
 - Fetches the salary of the employee with employee_id = 100 into v_salary.
- **Calculate New Salary:**
 - Adds the bonus to the retrieved salary.
- **Output:**
 - Displays the updated salary with the bonus added.

Looping and Conditional Statements

```
BEGIN
-- Loop through numbers 1 to 5
FOR i IN 1..5 LOOP
-- Check if the current number is 3
IF i = 3 THEN
-- Print a special message for the number 3
DBMS_OUTPUT.PUT_LINE('Special case for 3');
ELSE
-- Print the current number
DBMS_OUTPUT.PUT_LINE('Number: ' || i);
END IF;
END LOOP;
END;
```

- **Loop:**
 - Iterates from 1 to 5, with i taking each value in this range.
- **Condition:**
 - If i is 3, prints "Special case for 3".
 - For other values of i, prints "Number: " followed by the value of i.

Simple Examples for Each Feature (5/5)

Reduce Network Traffic

```
BEGIN
-- Insert a new employee with ID 101 and name 'John'
INSERT INTO employees (employee_id, first_name) VALUES (101, 'John');

-- Insert another employee with ID 102 and name 'Jane'
INSERT INTO employees (employee_id, first_name) VALUES (102, 'Jane');

-- Commit the transaction to save the changes
COMMIT;
END;
```

▪ Insert Statements:

- Adds two employees to the employees table: one with ID 101 and name 'John', and another with ID 102 and name 'Jane'.

▪ Commit:

- Saves both insertions in a single transaction, making the changes permanent.

▪ Batch Inserts in PL/SQL Block:

- Both INSERT statements are sent as a single request.
- A single commit operation finalizes all changes.

By consolidating operations into a single PL/SQL block, network traffic is optimized, leading to better performance and reduced latency.

Portable Application

```
-- This procedure will run on any Oracle Database regardless of OS
CREATE OR REPLACE PROCEDURE hello_world IS
BEGIN
-- Output a simple greeting message
DBMS_OUTPUT.PUT_LINE('Hello, World!');
END;
/
```

▪ Procedure:

- **CREATE OR REPLACE PROCEDURE hello_world IS:** Defines a new procedure named hello_world.
- **BEGIN:** Starts the executable part of the procedure.
- **DBMS_OUTPUT.PUT_LINE('Hello, World!');** Outputs the string "Hello, World!" to the console.
- **END;** Ends the procedure.

- **Execution:** This procedure can be executed in any environment where Oracle Database is installed, making it highly portable across different operating systems and versions of Oracle Database.

Common Challenges in PL/SQL Procedure Implementation and Solutions

```
CREATE OR REPLACE PROCEDURE Add_Numbers_With_Increment(  
    num1    IN  NUMBER,  
    num2    IN  NUMBER,  
    increment IN  NUMBER,  
    result  OUT NUMBER  
) AS  
BEGIN  
    -- Add the two numbers and the increment  
    result := (num1 + num2) + increment;  
  
    -- Output the result  
    DBMS_OUTPUT.PUT_LINE('The sum of ' || num1 || ' and ' || num2 ||  
        ' with an increment of ' || increment ||  
        ' is: ' || result);  
  
END;  
/
```

- num1 and num2 are the numbers to be added.
- increment is the value to be added to the sum of the two numbers.
- result is an output parameter that will hold the final result.
- DBMS_OUTPUT.PUT_LINE is used to print the result in the console.

Call the procedure

```
DECLARE  
    sum_result NUMBER;  
BEGIN  
    -- Call the procedure  
    Add_Numbers_With_Increment(5, 10, 2, sum_result);  
  
    -- Display the final result  
    DBMS_OUTPUT.PUT_LINE('Final Result: ' || sum_result);  
  
END;  
/
```

- The DECLARE block defines sum_result to store the output.
- The procedure Add_Numbers_With_Increment(5, 10, 2, sum_result) is called with:
 - 5 and 10 as the two numbers to add.
 - 2 as the increment.
 - sum_result to hold the result (17).
- The procedure computes $(5 + 10) + 2 = 17$ and stores it in sum_result.
- DBMS_OUTPUT.PUT_LINE prints the final result: "Final Result: 17".
- **The sum of 5 and 10 with an increment of 2 is: 17**
- **Final Result: 17**

Key Differences between Oracle SQL & PL/SQL

Feature	Oracle SQL	Oracle PL/SQL
Type	Declarative (Non-Procedural)	Procedural Language (Extension of SQL)
Purpose	Data querying and manipulation	Procedural logic and complex operations
Execution	Executes single statements	Executes blocks of code (multiple SQL statements and logic)
Control Structures	No control structures (loops, conditions)	Supports loops, conditions, exceptions
Reusability	No reusable logic	Supports reusable logic (procedures, functions)
Transaction Handling	Manages transactions (COMMIT, ROLLBACK)	Handles transactions within a block
Error Handling	Limited error handling	Advanced error handling using EXCEPTION blocks
Integration	Focuses solely on data manipulation	Integrates SQL within procedural code

Advantages of Oracle Database over other RDBMS systems

Category	Advantage
Scalability and Performance	- Horizontal and vertical scalability. - Real Application Clusters (RAC) for load balancing and high availability. - Advanced query optimization.
High Availability and Disaster Recovery	- Oracle Data Guard for failover and data protection. - Automatic Storage Management (ASM) for storage optimization. - Flashback Technology for quick data recovery.
Security	- Advanced encryption (in transit and at rest). - Virtual Private Database (VPD) for fine-grained access control. - Data masking and redaction.
Support for Advanced Data Types	- Support for XML, JSON, spatial data, and multimedia. - Object-relational model for complex data structures.
Cross-Platform Flexibility	- Runs on multiple platforms (Windows, Linux, Unix, Solaris). - Cloud compatibility (Oracle Cloud, AWS, Azure).
Enterprise-Level Features	- Partitioning for improved query performance. - Oracle Exadata for optimized performance. - Multitenant architecture for managing multiple databases.
Comprehensive Tooling and Ecosystem	- Oracle Enterprise Manager (OEM) for centralized monitoring and management. - Data integration and business intelligence tools.
Support and Community	- 24/7 enterprise-level support. - Active user community and extensive documentation.
Automation	- Autonomous database for automatic tuning, backups, and patching.
Compliance and Auditing	- Extensive auditing tools for regulatory compliance (GDPR, HIPAA, etc.).



Overview of the Oracle Database Environment (1/4)

Overview of the Oracle Database Environment

Oracle Database is a robust, enterprise-level relational database management system (RDBMS) designed to handle a wide range of data management tasks. It supports SQL and PL/SQL for querying and programming, and it is known for scalability, security, high availability, and extensive feature sets for various types of data workloads.

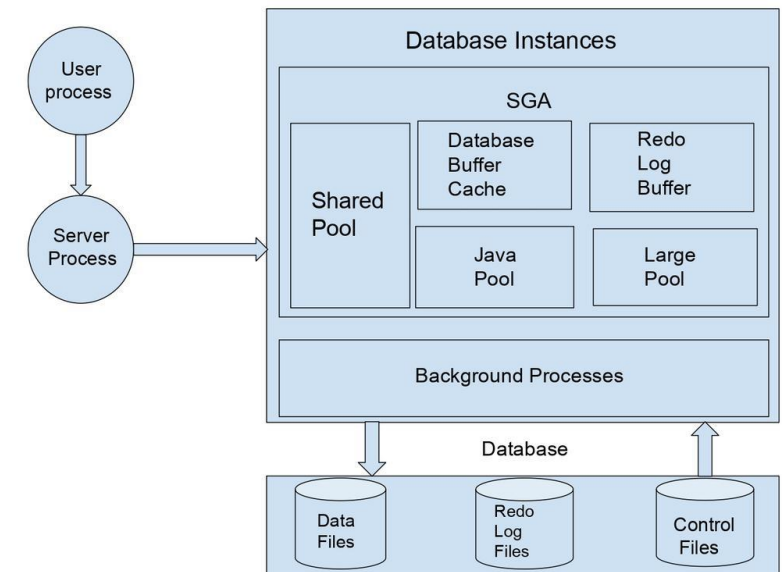
1. **Oracle Database Instance:** The combination of memory structures (SGA, PGA) and background processes that manage database operations.
2. **Physical Structures:** The files (data files, control files, redo log files) that physically store the database's data and metadata.
3. **Logical Structures:** Logical components like tablespaces, segments, extents, and blocks that organize how data is stored within the database.
4. **Schema Objects:** Database objects like tables, indexes, views, and sequences that define and store data within a schema.
5. **User and Security Management:** Roles, privileges, and users that control access to database objects and operations.
6. **Oracle Network Environment:** Tools like Oracle Net Services and listeners that manage communication between clients and the database.
7. **Oracle Multitenant Architecture (Optional):** Allows multiple pluggable databases (PDBs) to be housed within a single container database (CDB) for easier management.
8. **Oracle Data Guard (High Availability):** A high availability solution that replicates data to standby databases for disaster recovery.
9. **Oracle Backup and Recovery:** Tools like RMAN and Flashback that provide robust backup, restore, and point-in-time recovery options.
10. **Oracle RAC (Real Application Clusters):** A clustering solution that enables multiple instances to access a single database for scalability and high availability.
11. **Oracle Enterprise Manager (OEM):** A web-based tool for managing, monitoring, and configuring Oracle databases and infrastructure.

Overview of the Oracle Database Environment (2/4)

1. Oracle Database Instance

An **Oracle instance** consists of memory structures and background processes that allow data to be managed. It is composed of:

- **System Global Area (SGA):** A shared memory structure that contains data and control information for the database instance. It includes:
 - **Database Buffer Cache:** Stores copies of data blocks read from the data files.
 - **Shared Pool:** Caches SQL statements, data dictionary information, and execution plans.
 - **Redo Log Buffer:** Stores redo entries, which are used to recover the database in case of failure.
- **Program Global Area (PGA):** Private memory regions that contain data and control information for each server process.
- **Background Processes:** Include processes like DBWn (Database Writer), LGWR (Log Writer), SMON (System Monitor), and PMON (Process Monitor), which handle various system-level operations like writing data to disk, recovery, and memory management.



Source Image: <https://www.geeksforgeeks.org/oracle-architecture/>

Overview of the Oracle Database Environment (3/4)

Oracle Instance Diagram Explanation

A. User Process and Server Process:

- **User Process:** Represents a client-side application or process that interacts with the Oracle database. The user issues SQL queries or requests data.
- **Server Process:** Receives the request from the User Process and communicates with the Oracle Database to fetch or update data.

B. Database Instance:

An **Oracle Database Instance** is the combination of memory structures and background processes that are used to access data stored in the database. It primarily contains:

- **SGA (System Global Area):** This is a shared memory region that holds the instance's data and control information. It includes:
 - **Database Buffer Cache:** Temporarily stores copies of data blocks read from data files. When data is retrieved from the database, it is first stored here for quick access.
 - **Redo Log Buffer:** Temporarily stores changes made to the database. These changes are logged to ensure that data is not lost in case of a failure.
 - **Shared Pool:** Contains important data such as SQL queries, execution plans, and data dictionary cache, which help with parsing SQL statements and reusing frequently accessed queries.
 - **Java Pool:** Used for Java code and session-specific data within the Oracle database (if Java is used).
 - **Large Pool:** Used for larger memory allocations, like backup and restore operations.



Overview of the Oracle Database Environment (4/4)

C. Background Processes:

The Oracle database consists of multiple background processes that help manage tasks like writing data to disk, managing redo logs, and cleaning up the database. Examples include:

- **DBWn (Database Writer):** Writes modified (dirty) blocks from the buffer cache to the data files.
- **LGWR (Log Writer):** Writes redo log entries from the redo log buffer to the redo log files.
- **SMON (System Monitor):** Performs system recovery after a failure.

D. Database:

The database refers to the **physical files** where data is stored. These include:

- **Data Files:** Store the actual user data.
- **Redo Log Files:** Store information that can be used to recover data in case of a failure.
- **Control Files:** Contain metadata about the database structure, such as the location of data files and redo log files.

Overview of the Oracle Database Environment

2. Physical Structures

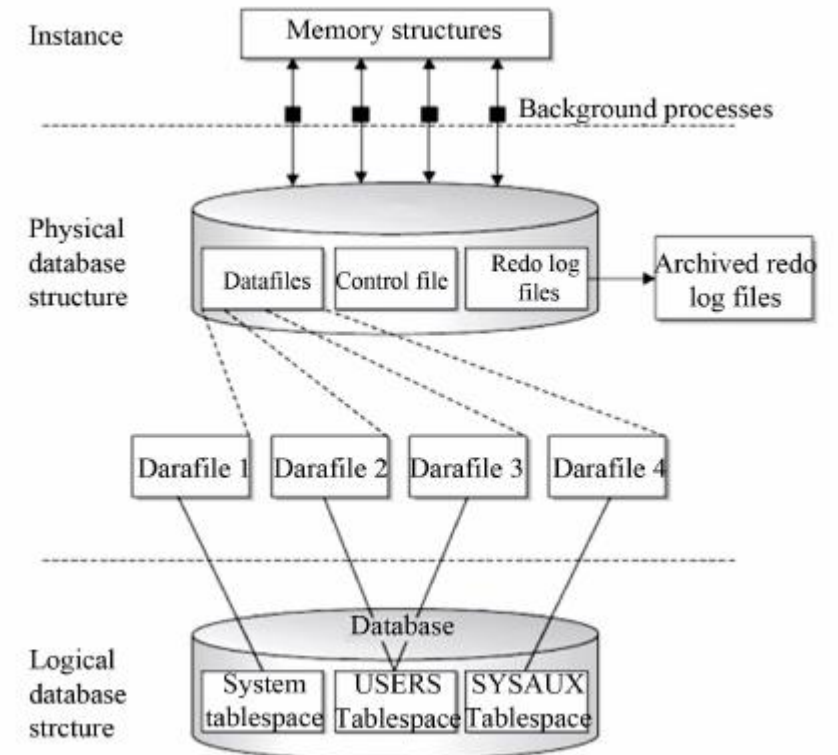
The physical storage of Oracle Database includes:

- **Data Files:** Store actual database data. These files are associated with tablespaces and contain table data, indexes, and other objects.
- **Control Files:** Contain metadata about the structure of the database and are critical for database startup and recovery.
- **Redo Log Files:** Store changes made to the database. They are essential for recovering the database in the event of a failure.
- **Archive Log Files:** A copy of redo log files generated in archive log mode, used for point-in-time recovery.

3. Logical Structures

Oracle uses logical structures to manage and organize data:

- **Tablespaces:** Logical storage units that contain one or more data files. Tablespaces allow data to be organized and managed independently.
- **Segments:** A segment is a set of extents that contain all of the data for a specific logical storage structure within a tablespace (e.g., table, index).
- **Extents:** A collection of contiguous data blocks that are allocated for storing a specific type of object (like a table or an index).
- **Data Blocks:** The smallest unit of I/O in the database, typically set to 8KB or 16KB.



Source Image: https://www.researchgate.net/figure/Oracle-physical-storage-structures_fig1_268349058

Overview of the Oracle Database Environment

4. Schema Objects

A **schema** is a logical collection of database objects owned by a user. Some key schema objects include:

- **Tables:** Store data in rows and columns. Tables are the primary object for holding user data.
- **Indexes:** Used to improve query performance by providing faster access to table rows.
- **Views:** Virtual tables created by a query that provides a dynamic result set of data from one or more tables.
- **Sequences:** Generate unique numeric values, often used for primary keys.
- **Synonyms:** Aliases for tables, views, or other database objects, which simplify object access.

5. User and Security Management

Oracle has a sophisticated security model based on user roles and privileges. Each database user has:

- **Users and Schemas:** Users are given access to database objects, and each user owns a schema containing objects like tables, views, and procedures.
- **Roles:** A collection of privileges that can be assigned to a user, making security easier to manage.
- **System Privileges:** Allow users to perform administrative tasks, such as creating tables or users.
- **Object Privileges:** Provide specific permissions on objects like tables (e.g., SELECT, INSERT, UPDATE, DELETE).

6. Oracle Network Environment

Oracle uses **Oracle Net Services** (formerly SQL*Net) to enable communication between the client application and the database over a network. Key components include:

- **Listeners:** A process that listens for client requests and establishes a connection to the database.
- **TNS (Transparent Network Substrate):** Manages the connections and addresses between Oracle clients and servers.
- **Database Links:** Enable queries and DML operations to be performed across different databases by linking one database to another.



Overview of the Oracle Database Environment

7. Oracle Multitenant Architecture (Optional)



In the Oracle 12c and later versions, the **multitenant architecture** enables the creation of a container database (CDB) that can hold multiple **pluggable databases (PDBs)**. This architecture improves consolidation, simplifies upgrades, and enables more efficient resource management.

8. Oracle Data Guard (High Availability)

Oracle Data Guard ensures high availability and disaster recovery for Oracle databases. It maintains one or more standby databases (copies of the primary database) that can be automatically or manually switched over if the primary database fails.

9. Oracle Backup and Recovery

Oracle provides several tools and features for backup and recovery, including:

- **RMAN (Recovery Manager):** Oracle's primary tool for backing up, restoring, and recovering databases.
- **Flashback Technology:** Allows administrators to quickly rewind the database to a previous point in time without requiring full recovery.

10. Oracle RAC (Real Application Clusters)

Oracle RAC allows multiple instances to access the same physical database, providing high availability and scalability. It enables load balancing and failover between instances to ensure continuous availability.

Overview of the Oracle Database Environment

11. Oracle Enterprise Manager (OEM)

OEM is Oracle's integrated management platform for monitoring and managing Oracle databases and other infrastructure. It provides a web-based interface to monitor performance, perform database administration, configure backups, and more.

Conclusion

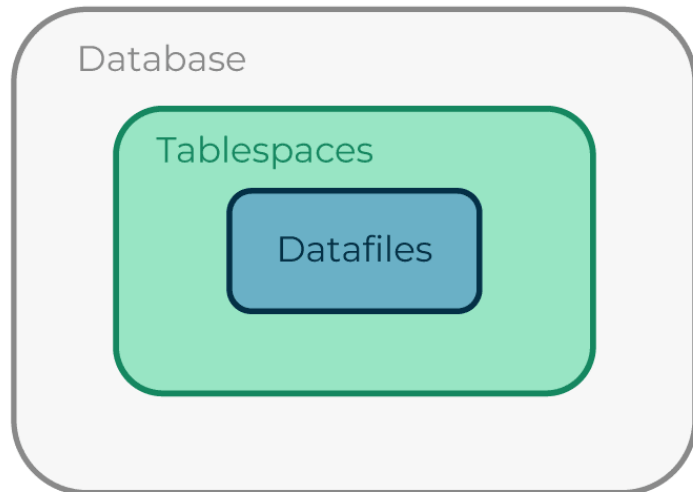
The **Oracle Database environment** is a comprehensive platform for managing structured data with a focus on performance, scalability, high availability, and security. It consists of a blend of physical and logical components, schema objects, security models, and tools that collectively support efficient data storage, retrieval, and management.

Introduction to Tablespaces, Datafiles, and Control Files

Tablespace in Oracle Database are logical storage units to store all the data. Logical storage units help users locate specific data and help in the retrieval of data.

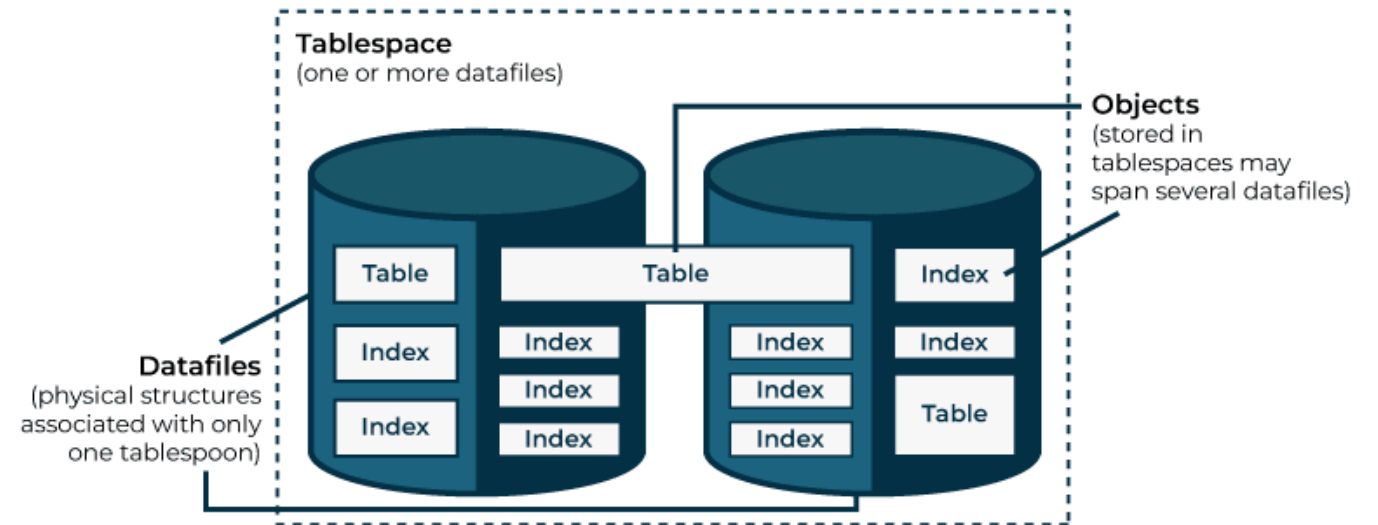
A database, as defined by Oracle, **is an organized collection of information** typically stored electronically in a computer system. The explanation is simple, however, it's not that simple to store the data.

The Database has a tablespace that stores all the data. Everything inside a database is stored in tablespace.



Type of Tablespace

- **SYSTEM Tablespace** – Oracle Database contains SYSTEM tablespace which is **automatically created when the database is created**. The tablespace contains a data file that stores data dictionary tables for the entire DB.
- **Undo Tablespaces** – These tablespaces **store the undo information and nothing else**.



Source Image: <https://www.geeksforgeeks.org/working-on-oracle-tablespaces-for-developers/>

Relationship Between PL/SQL Developer and Oracle Tablespaces

While the creation and extension of tablespaces is typically the responsibility of the DBA, Oracle PL/SQL developers may encounter situations where tablespace issues arise. A basic understanding of tablespaces can help developers address performance problems and reduce the burden on DBAs.

Issues with Oracle Tablespaces

Tablespaces come with a standard size and can eventually become full. When nearly or completely full, the following issues may occur:

- Developers cannot create new tables.
- Developers cannot create any other Oracle objects.
- Data cannot be inserted into existing tables.

DBA's Part of the Solution

When a tablespace is full, the DBA typically extends it using the following SQL statement:

```
ALTER TABLE Tablespace_Name ADD DATAFILE 'Data_File_Location' SIZE 40M AUTOEXTEND ON;
```

In some cases, the DBA may create a new tablespace and ask developers to move their data from the old tablespace to the new one.

Relationship Between PL/SQL Developer and Oracle Tablespaces

PL/SQL Developer's Role

Developers also play a role in maintaining efficient performance related to tablespaces. Some actions that can be taken by developers include:

1. Move an Existing Table to a New Tablespace

If needed, a developer can move a table to a new tablespace:

```
ALTER TABLE <TABLE_NAME> MOVE TABLESPACE <NEW_TABLESPACE_NAME>;
```

Since moving a table invalidates its indexes, the following command should be run to rebuild them:

```
ALTER INDEX <INDEX_NAME> REBUILD;
```

2. Create a Backup Table in a New Tablespace

Developers can also create backups into new tablespaces:

```
CREATE TABLE <BACKUP_TABLE_NAME> TABLESPACE <NEW_TABLESPACE_NAME> AS SELECT * FROM <TABLE_FOR_BACKUP>;
```

3. Create Indexes in a New Tablespace

Developers may create indexes in new tablespaces as needed:

```
CREATE INDEX <INDEX_NAME> ON <TABLE_NAME> (COLUMNS ...) TABLESPACE <NEW_TABLESPACE_NAME>;
```

Relationship Between PL/SQL Developer and Oracle Tablespaces

Best Practices

It's a good practice for PL/SQL developers to explicitly specify the tablespace when creating tables, indexes, and other objects. Experienced developers have noted that new developers often create temporary or normal tables without specifying the tablespace, which can lead to performance issues later.

List All Tablespaces

```
SELECT tablespace_name,  
       status,  
       contents,  
       extent_management,  
       allocation_type  
FROM dba_tablespaces;
```

PL/SQL Disadvantage



Licensing Costs

- Oracle's PL/SQL tools are expensive, making it costly for smaller organizations.

High Memory Usage

- Stored procedures in PL/SQL can use a lot of memory, which may slow down the system.

Limited Web Support

- PL/SQL is mainly for database tasks and has limited capabilities for building modern web applications compared to languages like PHP or Python.

Maintenance Issues

- Changes in the database often require updates in the code, leading to more maintenance work.

Difficult Debugging

- PL/SQL lacks advanced debugging tools, making it harder to find and fix errors.

Steep Learning Curve

- PL/SQL is difficult for beginners to learn quickly due to its complex syntax and structure.

Database Dependence

- SQL works only with Oracle databases, so it can't be used with other databases or platforms easily.

Conclusion: Key Takeaways PL/SQL



- What is the difference between PL/SQL and SQL?
- Why would you choose PL/SQL over SQL in certain scenarios?
- Are there situations where using PL/SQL is not recommended? If so, provide examples.
- What are the advantages of using PL/SQL?
- What are the disadvantages of using PL/SQL?
- Can you think of a scenario where PL/SQL would significantly improve the performance or functionality of a database application compared to using SQL alone? Describe the scenario.
- What potential challenges might arise when maintaining PL/SQL code compared to SQL?
- In what cases would using plain SQL be more efficient or preferable over PL/SQL?

Thank you!